프로그래밍I
탐색 기반 알고리즘

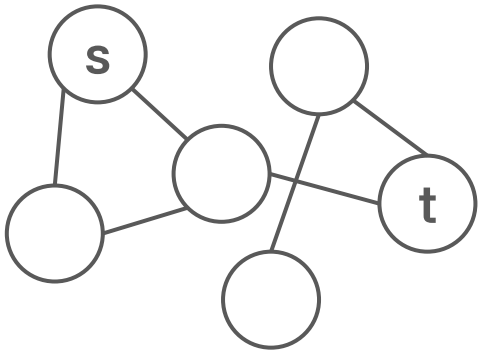
03 비선형 구조의 탐색(1)



01 비선형 구조

- **비선형 구조**
  - i 번째 원소를 탐색한 다음 그 원소와 연결된 다른 원소를 탐색하려고 할 때, 여러 개의 원소가 존재하는 탐색구조
  - 일반적으로, 자료가 트리나 그래프로 구성되며, 단순히 반복문을 이용하여 탐색하기 어려운 관계로 스택이나 큐와 같은 자료구조를 활용하여 탐색

O	정점 vertex 노드 node	- 자료를 나타내는 원
-	간선 edge 링크 link	- 양방향 연결은 – 또는 ↔ 사용 - 단방향 연결은 → 사용 - 가중치는 연결 상태와 함께 표현 가능

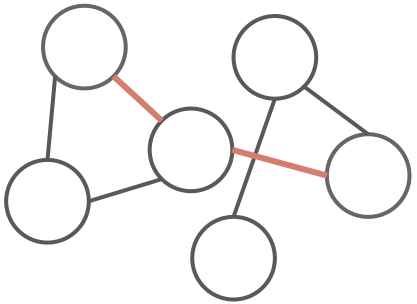


Copyright ⓒ 2026 by Lee San. All rights reserved.

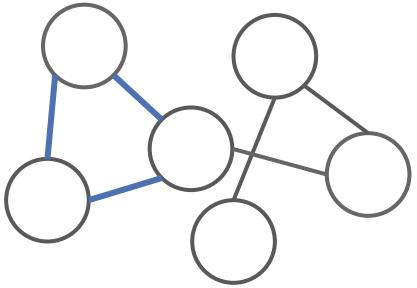
01 비선형 구조

- **그래프**
  - 데이터가 정점과 간선으로 구성되며 자유롭게 연결된 구조

경로 path 는 임의의 정점 s에서 임의의 정점 t로 이동할 때, s에서 t로 이동하는데 사용한 정점들을 연결하고 있는 간선들의 순서로 된 집합



회로 cycle 는 그래프에서 임의의 정점 s에서 동일한 정점 s로의 경로

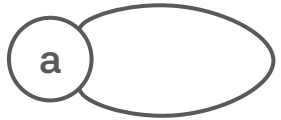


Copyright ⓒ 2026 by Lee San. All rights reserved.

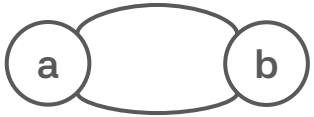
01 비선형 구조

- **그래프**
  - 데이터가 정점과 간선으로 구성되며 자유롭게 연결된 구조

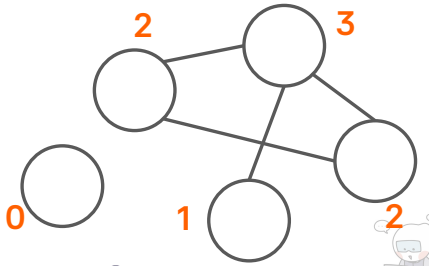
자기 간선 loop 이란 임의의 정점에서 자기 자신으로 연결하고 있는 경우



다중 간선 multi edge 이란 임의의 정점에서 다른 정점으로 연결된 간선의 수가 2개 이상일 경우



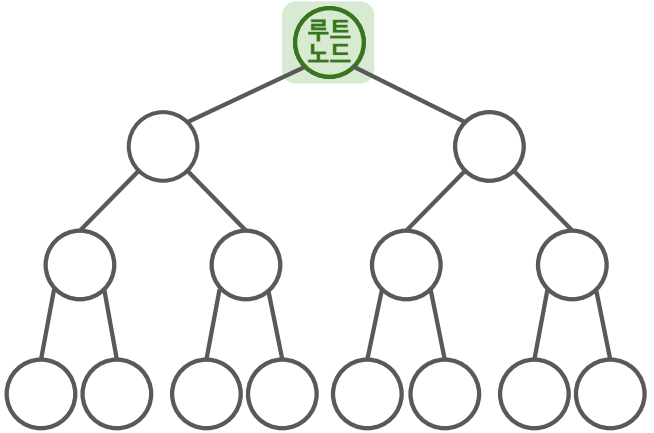
그래프의 차수 degree 란 그래프의 임의의 한 정점에서 다른 정점으로 연결된 간선의 수



Copyright ⓒ 2026 by Lee San. All rights reserved.

01 비선형 구조

- **트리**
  - 그래프의 일종으로, 데이터가 부모-자식 관계를 이루며 계층적으로 연결된 구조
  - 하나의 시작점인 루트 노드에서 아래로 뻗어 나가며, 회로가 없는 형태

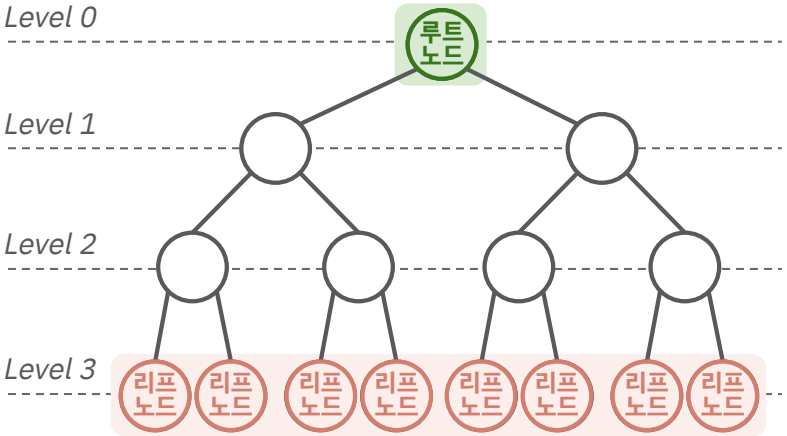


Copyright ⓒ 2026 by Lee San. All rights reserved.

01 비선형 구조

- **트리**

루트노드 Root Node	최상위 부모노드
부모노드 Parent Node	다른 노드로 연결되어 나가는 노드
자식노드 Child Node	다른 노드로부터 연결된 노드
형제노드 Sibling Node	같은 부모를 가진 자식노드
리프노드 Leaf Node	자식이 없는 노드
깊이 Level	루트노드로부터 그 위치까지 경로 길이
높이 Depth	루트노드부터 가장 깊은 노드까지 경로 길이
차수 Degree	자식 노드의 개수



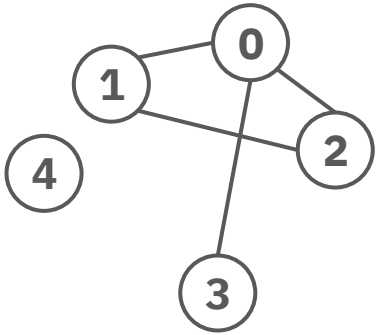
▲ 높이가 3인 트리 예시

Copyright ⓒ 2026 by Lee San. All rights reserved.

02 그래프 구현 방식

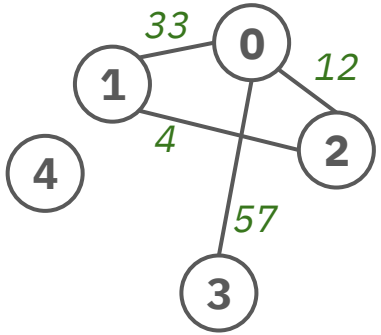
- **인접행렬** Adjacency Matrix
  - 그래프의 정점들을 행과 열에 각각 배치하고, 두 정점이 연결되어 있는지를 행렬의 값으로 나타내는 방식

< 가중치가 없는 표현 예시 >



	0	1	2	3	4
0	0	1	1	1	0
1	1	0	1	0	0
2	1	1	0	0	0
3	1	0	0	0	0
4	0	0	0	0	0

< 가중치가 있는 표현 예시 >



	0	1	2	3	4
0	0	33	12	57	0
1	33	0	4	0	0
2	12	4	0	0	0
3	57	0	0	0	0
4	0	0	0	0	0

Copyright ⓒ 2026 by Lee San. All rights reserved.

02 그래프 구현 방식

- **인접행렬로 그래프를 구현하는 방법**

```c
#include <stdio.h>
int vertex_cnt, edge_cnt, graph[101][101];

int main(){
  scanf("%d %d", &vertex_cnt, &edge_cnt);
  for(int i=0; i<edge_cnt; i++){
    int a, b, weight;
    scanf("%d %d %d", &a, &b, &weight);
    graph[a][b] = graph[b][a] = weight;
  }
}
```

첫 번째 줄에 정점의 수와 간선의 수가 공백으로 구분되어 입력된다.

두 번째 줄부터 간선으로 연결된 두 정점의 번호와 가중치가 공백으로 구분되어 입력된다.

Copyright ⓒ 2026 by Lee San. All rights reserved.

02 그래프 구현 방식

- **인접리스트** Adjacency List
  - 그래프에서 각 정점에 연결된 정점들을 목록으로 저장하는 방식
  - 인접 행렬은 연결되지 않은 부분(0)까지 모두 표현하기 때문에 탐색 효율이 떨어지는 경우가 많은데, 이러한 단점을 극복하기 위하여 제안된 방법

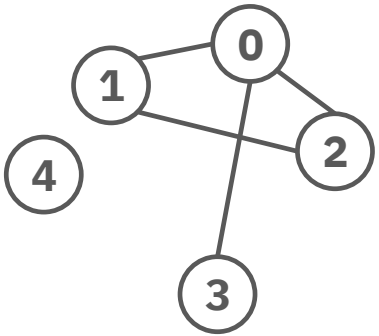
Copyright ⓒ 2026 by Lee San. All rights reserved.

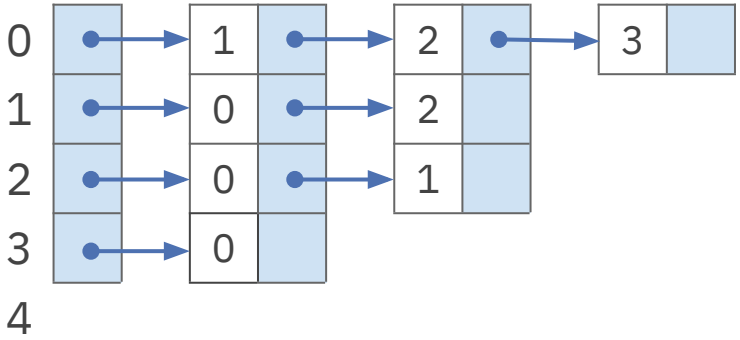
02 그래프 구현 방식

- 인접리스트 Adjacency List

연결 리스트를 활용한 표현 | 배열, 벡터를 활용한 표현

< 가중치가 없는 표현 예시 >





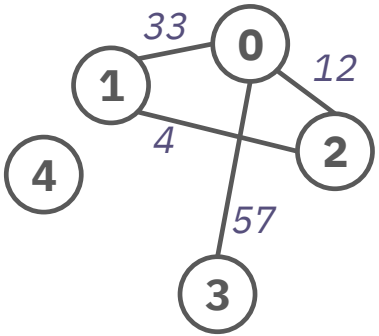
Copyright ⓒ 2026 by Lee San. All rights reserved.

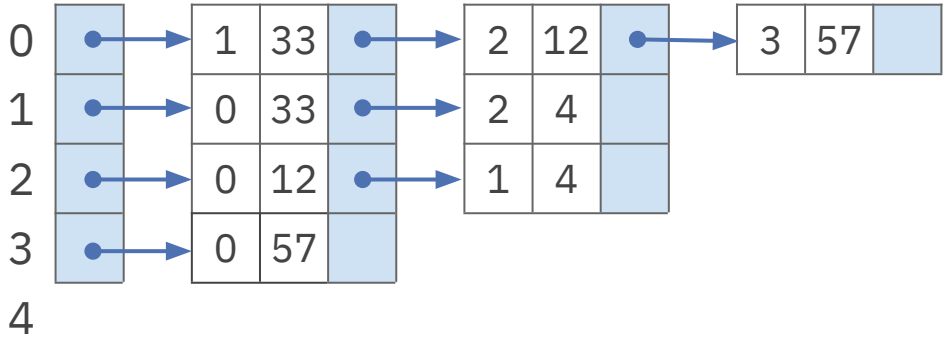
02 그래프 구현 방식

- 인접리스트 Adjacency List

연결 리스트를 활용한 표현 | 배열, 벡터를 활용한 표현

< 가중치가 있는 표현 예시 >





Copyright ⓒ 2026 by Lee San. All rights reserved.

02 그래프 구현 방식

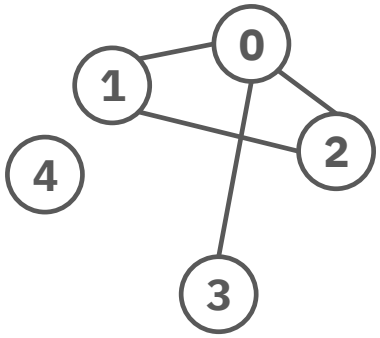
- 인접리스트 Adjacency List

C++의 vector는 크기가 자동으로 늘어나는 동적 배열

연결 리스트를 활용한 표현

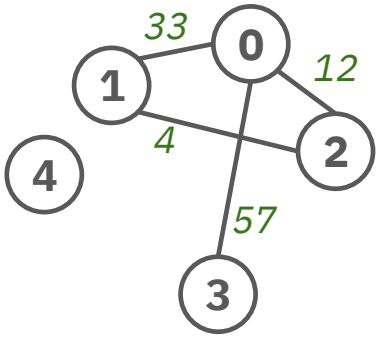
배열, 벡터를 활용한 표현

< 가중치가 없는 표현 예시 >



0	1	2	3
1	0	2	
2	0	1	
3	0		
4			

< 가중치가 있는 표현 예시 >



0	1	33	2	12	3	57
1	0	33	2	4		
2	0	12	1	4		
3	0	57				
4						

Copyright ⓒ 2026 by Lee San. All rights reserved.

02 그래프 구현 방식

- **그래프 구현 방식 정리**

단일값을 저장하는 방식 -> 자료형 배열명[크기];
하나의 정점에 대응하는 값이 하나뿐이므로, 인접 정점이 복수이면 표현이 어려움.

인접행렬 방식 -> 자료형 배열명[크기][크기];
모든 정점 쌍에 대한 기억 공간을 사전에 확보함. 간선이 존재하지 않는 위치도 공간을 점유하므로, 정점 수가 증가하면 공간 낭비가 심함.

인접리스트 방식 → vector<자료형> 배열명[크기];
각 정점에 가변 길이의 목록을 배정하여 실제로 존재하는 간선만 저장함.
공간의 낭비가 없으며, 인접 정점을 순회하는 시간도 단축됨.

Copyright ⓒ 2026 by Lee San. All rights reserved.

02 그래프 구현 방식

- 인접리스트로 그래프를 구현하는 방법 – 벡터 활용

```cpp
#include <stdio.h>
#include <vector>
using namespace std;

// graph[정점] = {인접 정점, 가중치} 목록
vector< pair<int, int> > graph[101];
```

...
graph[1] → (2,47) (3,69)
graph[2] → (1,47) (4,57) (5,124)
graph[3] → (1,69) (4,37) (5,59) (6,86)
graph[4] → (2,57) (3,37) (6,27) (7,94)
...

Copyright ⓒ 2026 by Lee San. All rights reserved.

02 그래프 구현 방식

- 인접리스트로 그래프를 구현하는 방법 – 벡터 활용

```
int main() {
    int vertex_cnt, edge_cnt;
    scanf("%d %d", &vertex_cnt, &edge_cnt);

    for(int i=0; i<edge_cnt; i++) {
        int a, b, weight;
        scanf("%d %d %d", &a, &b, &weight);

      // a->b 간선 저장(push_back: 맨 뒤에 데이터 추가)
        graph[a].push_back({b, weight});

      // b->a간선 저장(push_back: 맨 뒤에 데이터 추가)
        graph[b].push_back({a, weight});
    }
}
```

```
7 11
1 2 47
1 3 69
2 4 57
2 5 124
3 4 37
3 5 59
3 6 86
4 6 27
4 7 94
5 7 21
6 7 40
```

Copyright ⓒ 2026 by Lee San. All rights reserved.

02 그래프 구현 방식

vector에서 자주 쓰는 함수

함수	설명	사용 예시
push_back()	맨 뒤에 데이터 추가	v.push_back(10);
pop_back()	맨 뒤 데이터 삭제	v.pop_back();
empty()	비어 있는지 확인	v.empty();
clear()	모든 데이터 삭제	v.clear();
size()	저장된 데이터 개수 반환	v.size();
front()	첫 번째 데이터 반환	v.front();
back()	마지막 데이터 반환	v.back();
begin()	첫 번째 위치 반환	v.begin();
end()	마지막 다음 위치 반환	v.end();

Copyright ⓒ 2026 by Lee San. All rights reserved.

SASA OJ에서 다음 문제를 풀어볼까요?

[2026_2학기_프로그래밍I]

🔎 탐색기반설계 연습문제 #2



Copyright ⓒ 2026 by Lee San. All rights reserved.